

プロジェクト定義書: TaskNavi

1. アプリケーションの概要

TaskNavi は、業務やプロジェクトの進行管理を行うための Java Swing アプリケーションです。

主な対象は、WBS（作業分解構造）・カンバン・ガントチャートを組み合わせて、タスクの作成、編集、進捗管理、検索、優先度管理を一枚の画面で扱えるようにしたものです。

このアプリが解決する課題:

- タスクの一覧を見通しよく管理したい
- 進捗・担当者・期間を視覚的に把握したい
- 優先度ごとに重要度を見分けたい
- 画面ごとに異なる見方（ツリー・ボード・日程表）で管理したい
- ローカル環境で簡単にデータを保持したい

提供する主な機能:

- ユーザー登録 / ログイン / パスワード再設定
- WBS 形式のタスク木構造管理
- カンバン形式でのステータス移動
- ガントチャート形式での期間表示
- 検索とフィルタ
- 優先度色分け
- SQLite によるデータ保存
- CSV / JSON 形式でのインポート / エクスポート

2. システム構成・ファイル一覧

2.1 システム構成の概要

TaskNavi は、以下の層で構成されています。

- UI 層: Swing の JFrame / JPanel / JTree / JTable 風コンポーネント
- アプリ制御層: Main, MainFrame, LoginFrame など
- ドメイン層: Task
- データアクセス層: TaskDao, UserDao
- DB 層: DatabaseUtil, Database
- ユーティリティ層: CsvUtil, EmailUtil, VerificationManager, SampleDataUtil

2.2 Java ファイル一覧と役割

ファイル	クラス名	役割
src/main/java/org/example/Main.java	Main	アプリケーションの開始点。初期化とログイン画面表示
src/main/java/org/example/LoginFrame.java	LoginFrame	ログイン画面。ユーザー認証の入口
src/main/java/org/example/RegisterFrame.java	RegisterFrame	新規ユーザー登録画面
src/main/java/org/example/ResetPasswordFrame.java	ResetPasswordFrame	パスワード再設定画面
src/main/java/org/example/MainFrame.java	MainFrame	メイン画面。WBS / カンバン / ガントをタブで切替
src/main/java/org/example/AppTheme.java	AppTheme	色、フォント、ボタン、入力欄の共通デザイン定義
src/main/java/org/example/Task.java	Task	タスクのデータモデル。ID、名前、期限、優先度などを保持
src/main/java/org/example/TaskDao.java	TaskDao	タスクの CRUD と親タスク進捗再計算
src/main/java/org/example/UserDao.java	UserDao	ユーザー登録・認証・パスワード更新
src/main/java/org/example/UserSession.java	UserSession	ログイン中ユーザー情報の保持
src/main/java/org/example/TaskDialog.java	TaskDialog	タスク追加/編集のダイアログ
src/main/java/org/example/WbsPanel.java	WbsPanel	WBS 表形式のツリー管理画面
src/main/java/org/example/KanbanPanel.java	KanbanPanel	カンバンボード画面
src/main/java/org/example/GanttPanel.java	GanttPanel	ガントチャート画面
src/main/java/org/example/Database.java	Database	SQLite への接続と初期化処理
src/main/java/org/example/util/DatabaseUtil.java	DatabaseUtil	SQLite 接続とテーブル作成 / マイグレーション
src/main/java/org/example/CsvUtil.java	CsvUtil	CSV / JSON のエクスポート / インポート
src/main/java/org/example/EmailUtil.java	EmailUtil	認証コードをメール送信する処理

ファイル	クラス名	役割
src/main/java/org/example/VerificationManager.java	VerificationManager	認証コードの発行・照合・破棄
src/main/java/org/example/SampleDataUtil.java	SampleDataUtil	初回起動時のサンプルデータ投入

2.3 主要ファイルの責務

- Main
 - プログラム開始時にデータベースを初期化し、UI を起動する
- MainFrame
 - アプリ全体の枠組み。3 つの画面を切り替える
- WbsPanel
 - タスクの木構造管理と詳細編集フォーム
- KanbanPanel
 - ステータス別ボード表示
- GanttPanel
 - 日程と期間をグラフィカルに表示
- TaskDao
 - SQLite と Java の Task オブジェクトをつなぐ中核
- DatabaseUtil
 - tasks / users テーブルの自動生成と更新
- Task
 - 1 タスクの属性を表すモデル

3. 主要な機能の仕様（画面とデータ）

3.1 WBS 画面の仕様

WbsPanel でできること:

- タスクをツリー形式で表示
- 親子関係でタスクを整理
- タスク名検索
- ステータスでフィルタ
- プロジェクト単位で絞り込み
- フィルタクリア
- 新規プロジェクト作成
- 新規サブタスク追加
- タスク複製
- タスク削除
- 既存タスクの選択と編集
- 優先度の指定

- 開始日 / 終了日 / 担当者 / ステータスの編集

検索とフィルタのルール:

- キーワード検索: タスク名に含まれる文字列を対象
- ステータスフィルタ: 「すべて」「未着手」「進行中」「完了」
- プロジェクトフィルタ: 親子関係で対象プロジェクト配下のみ表示
- フィルタクリア: 現在の検索条件と状態フィルタを解除

優先度の色分けルール:

- 高: 赤
- 中: 青
- 低: 灰色
- この優先度はツリー表示・カンバンカード・ガントバーに反映される
- AppTheme で共通定義されており、見た目の一貫性を保っている

3.2 カンバン画面の仕様

KanbanPanel でできること:

- ステータスごとに 3 列を表示
 - 未着手
 - 進行中
 - 完了
- タスクカードを横並びで表示
- 検索キーワードでカードを絞り込み
- 左右のボタンでステータスを前後移動
- 優先度バーをカードに表示

3.3 ガントチャート画面の仕様

GanttPanel でできること:

- タスク期間を横棒で表示
- 日付単位でスケジュールを視覚化
- WBS の階層順に行を並べる
- ホバー時に詳細ツールチップ表示
- 進捗率と優先度をバーの見た目や tooltip で表現
- マウスホイールで日幅を拡大・縮小

3.4 データベース仕様

SQLite を中心に使用し、主に次の 2 つのテーブルを扱う。

users テーブル

保存項目:

- id
- username
- password
- email
- role

用途:

- ログイン認証
- ユーザー情報管理
- 管理者/一般ユーザー区分

tasks テーブル

保存項目:

- id
- parent_id
- level
- name
- assignee
- start_date
- end_date
- progress
- status
- priority

用途:

- タスク本体の保存
- 親子関係の管理
- 進捗と期間管理
- 優先度の保存

優先度の値:

- 高
- 中
- 低

データベース側の補正:

- DatabaseUtil.initializeDatabase() で tasks テーブルに priority カラムがなければ追加
- 既存データの互換性を保つための ALTER TABLE 対応あり

3.5 CSV / JSON 連携

CsvUtil により以下の形式を扱う:

- CSV

- BOM 付き UTF-8 CSV
- TSV
- JSON

活用法:

- タスク一覧の出力
- 他システムへの移行
- バックアップと復元

4. プログラムの処理の流れ（ライフサイクル）

4.1 起動から画面表示までの流れ

1. Main.main() が実行される
 - src/main/java/org/example/Main.java
 - Java アプリのエントリーポイント
2. 初回起動時にサンプルデータを投入
 - SampleDataUtil.insertSampleTasksIfEmpty()
 - tasks テーブルが空なら、見本タスクを登録する
3. Swing の GUI を EDT 上で開始
 - SwingUtilities.invokeLater(...)
 - UI はイベントディスパッチスレッドで動かす設計になっている
4. ログイン画面を表示
 - LoginFrame が生成され、setVisible(true) で表示される
5. ログイン処理
 - username / password を UserDao.authenticate() に渡す
 - users テーブルからハッシュ化済みパスワードを照合
 - 成功すれば UserSession にログイン状態を保存
6. メイン画面を表示
 - MainFrame が起動
 - WBS / カンバン / ガントの 3 つのタブを持つコンテナを生成
7. 各画面がデータを読み込む
 - WbsPanel.refreshWbs()
 - KanbanPanel.refreshKanban()
 - GanttPanel.paintComponent()
 - 各画面で TaskDao.getAllTasks() を呼ぶ
8. DB からタスク一覧を取得
 - TaskDao.getAllTasks()
 - SQLite の tasks テーブルから Task オブジェクトのリストを生成
 - priority など読み出される
9. ユーザーが編集や追加を行う
 - WbsPanel でフォーム編集
 - TaskDao.addTask() / updateTask() で DB に反映
 - 変更後に画面再描画を行う

10. 実行結果が再描画される

- 修正後のデータが再読み込みされ、WBS / カンバン / ガントの各ビューが最新状態に更新される

4.2 画面間連携のイメージ

Main → LoginFrame → UserDao → UserSession → MainFrame → WbsPanel / KanbanPanel / GanttPanel → TaskDao
→ DatabaseUtil → SQLite

4.3 重要な設計思想

- UI とデータアクセスを分離している
- TaskDao を通して DB 操作をまとめている
- Task はデータモデルとして振る舞う
- AppTheme は見た目を一元管理する
- priority は DB と UI に横断的に渡る重要属性として扱われている

この文書はプロジェクトの定義書として扱えるように記載しています。